

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería
SI3006 — Seguridad de la Información

Reporte de Pruebas de Seguridad Ofensivas

*Explotación y Remediación de una Vulnerabilidad de Inyección SQL en OWASP
Juice Shop*

Aplicación bajo prueba: OWASP Juice Shop v17.x
Tipo de prueba: Caja blanca (White Box Testing)

Equipo de Trabajo

Camila Vélez
María Acevedo
Alejandro Garcés
Alejandro Arteaga
Jean Carlo Londoño

Medellín, Colombia — Mayo de 2026

1. Resumen Ejecutivo

El presente reporte documenta el descubrimiento, explotación y remediación de una vulnerabilidad de inyección SQL identificada en el componente de autenticación de la aplicación web OWASP Juice Shop. La prueba se realizó en un entorno controlado bajo metodología de caja blanca, con acceso al código fuente, sobre una instancia desplegada por el equipo en Amazon Web Services (AWS Academy). El hallazgo permite a un atacante no autenticado eludir completamente el mecanismo de inicio de sesión y obtener acceso administrativo a la aplicación, comprometiendo la confidencialidad, la integridad y la trazabilidad de las operaciones del sistema.

La vulnerabilidad ha sido clasificada con una severidad CRÍTICA conforme al estándar CVSS v3.1 (puntuación 9.8) y se mapea con la categoría A03:2021 — Injection del OWASP Top 10 y con el identificador CWE-89 (Improper Neutralization of Special Elements used in an SQL Command). La remediación propuesta sustituye la concatenación de cadenas en la consulta SQL por consultas parametrizadas con bind variables, aprovechando las capacidades nativas del ORM Sequelize ya presente en el proyecto.

2. Alcance y Metodología

2.1 Tipo de prueba

La auditoría corresponde a una prueba de seguridad ofensiva de tipo caja blanca (white box). El equipo evaluador contó con acceso completo al código fuente del repositorio público de OWASP Juice Shop (<https://github.com/juice-shop/juice-shop>), lo que permitió combinar técnicas de análisis estático manual con explotación dinámica desde la interfaz pública.

2.2 Propósito

El propósito de la prueba es validar la presencia de una vulnerabilidad explotable en el flujo de autenticación, documentar su impacto sobre los pilares de seguridad de la información (CIA) y proponer una remediación a nivel de código que pueda ser integrada al pipeline de desarrollo de la aplicación.

2.3 Alcance

- URL bajo prueba: `http://<IP_EC2>:3000` (instancia desplegada en AWS Academy).
- Versión de la aplicación: OWASP Juice Shop v17.x, contenedor oficial `bkimminich/juice-shop`.
- Componente evaluado: endpoint `POST /rest/user/login` (módulo de autenticación).
- Quedan fuera del alcance los demás módulos de la aplicación, así como la infraestructura subyacente (red, sistema operativo, contenedor Docker).

2.4 Metodología

Se siguió un procedimiento de cuatro fases alineado con el ciclo de pruebas de penetración de OWASP Web Security Testing Guide (WSTG):

- Reconocimiento: identificación de los endpoints expuestos por la aplicación e inspección del código fuente del repositorio.

- Análisis estático: revisión manual del archivo routes/login.ts para detectar patrones inseguros de construcción de consultas SQL.
- Explotación: envío de payloads de inyección SQL desde el formulario de login y desde Burp Suite Community Edition para confirmar la vulnerabilidad.
- Remediación y verificación: implementación de la corrección en código y reintento del mismo payload para validar que el ataque ya no es viable.

2.5 Criterios de evaluación de riesgo

La severidad de los hallazgos se calcula con CVSS v3.1, complementada con la categorización OWASP Top 10 2021 y CWE para facilitar su trazabilidad con catálogos públicos. La probabilidad de explotación se estima a partir de la complejidad del ataque, la disponibilidad de payloads públicos y la madurez de las herramientas automatizadas que detectan este tipo de fallas (sqlmap, OWASP ZAP, etc.).

3. Narrativa del Ataque

Tras desplegar la aplicación, el evaluador navegó hasta el formulario de inicio de sesión disponible en la ruta `/#/login`. Se interceptó la petición con Burp Suite y se identificó que el endpoint backend invocado es `POST /rest/user/login`, recibiendo un cuerpo JSON con los campos email y password.

La revisión del archivo routes/login.ts en el código fuente reveló que la consulta SQL para validar las credenciales se construye mediante concatenación directa de cadenas con el contenido del campo email proveniente del usuario, sin escape ni parametrización. Este patrón es la causa raíz de la vulnerabilidad.

Para confirmar la explotación, se envió como email el payload `" ' OR 1=1-- "` (con espacio final, propio de SQLite) y un password arbitrario. La aplicación respondió con un token JWT válido y la sesión iniciada corresponde al primer usuario de la tabla, que en Juice Shop es la cuenta administradora `admin@juice-sh.op`. Con esta sesión se accedió al panel `/administration`, normalmente protegido, demostrando una elevación de privilegios completa.

4. Matriz Resumen de Vulnerabilidades

ID	Hallazgo	Severidad	CVSS v3.1	OWASP	CWE
VULN-001	Inyección SQL en el endpoint de autenticación <code>POST /rest/user/login</code> que permite eludir el mecanismo de login.	Crítica	9.8	A03:2021 — Injection	CWE-89

5. Detalle de la Vulnerabilidad — VULN-001

5.1 Identificación

- Identificador interno: VULN-001
- Nombre: SQL Injection en autenticación (Authentication Bypass)

- Categoría OWASP Top 10 2021: A03:2021 — Injection
- CWE: CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)
- CVSS v3.1: 9.8 — CRÍTICA. Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
- Endpoint afectado: POST /rest/user/login
- Archivo en código fuente: routes/login.ts

5.2 Descripción

La función que valida las credenciales del usuario construye la sentencia SQL como una cadena en la que se inserta directamente el contenido del campo email recibido del cliente. La aplicación no aplica ninguna forma de parametrización, escape o validación restrictiva sobre la entrada antes de pasarla al motor SQLite. En consecuencia, un atacante que controle el contenido del campo email puede inyectar fragmentos arbitrarios de SQL que el motor ejecutará como parte legítima de la consulta.

El payload mínimo de explotación, conocido públicamente y enseñado en cualquier curso introductorio de seguridad web, es:

```
' OR 1=1--
```

Al inyectarlo en el campo email, la consulta resultante adopta una forma equivalente a:

```
SELECT * FROM Users WHERE email = '' OR 1=1--' AND password = '...' AND
deletedAt IS NULL
```

La cláusula 1=1 hace que la condición sea siempre verdadera y los dos guiones convierten el resto de la consulta en un comentario, de modo que la verificación del password se elimina por completo. SQLite devuelve la primera fila de la tabla Users y la aplicación emite un token JWT válido para esa cuenta, que en la base de datos por defecto es el usuario administrador.

5.3 Superficie de ataque

La vulnerabilidad es accesible sin autenticación previa, sin necesidad de ingeniería social y sin requerir interacción del usuario legítimo. Es alcanzable directamente desde Internet siempre que el puerto 3000 (o el reverse proxy correspondiente) esté expuesto. Cualquier cliente HTTP es suficiente para explotarla: un navegador, curl, Postman, Burp Suite o sqlmap. La complejidad técnica es baja y existen herramientas automatizadas que descubren este tipo de fallas en cuestión de minutos.

5.4 Impacto

Sobre los pilares CIA:

- Confidencialidad: el atacante obtiene acceso a la cuenta administradora y, a través del panel /administration, puede consultar la lista completa de usuarios, pedidos y mensajes de retroalimentación.
- Integridad: con privilegios de administrador, el atacante puede modificar pedidos, alterar precios, borrar reseñas y manipular cualquier dato persistente de la aplicación.
- Disponibilidad: la cuenta administradora permite eliminar usuarios y productos, así como degradar el servicio mediante operaciones masivas sobre la base de datos.

Sobre el negocio:

- Pérdida de la trazabilidad: las acciones realizadas se registran a nombre del administrador legítimo, dificultando la atribución forense.

- Exposición regulatoria: la fuga de datos personales de los clientes activa los deberes de notificación previstos por leyes como la Ley 1581 de 2012 (Colombia) y el GDPR para clientes europeos.
- Daño reputacional: una explotación pública del fallo desincentiva la confianza del cliente y atrae cobertura mediática negativa.

5.5 Evidencia

La evidencia gráfica de la explotación se incluye en la presentación complementaria a este informe. Incluye:

- Captura del formulario de login antes del ataque, con credenciales legítimas inválidas.
- Captura de la petición POST /rest/user/login interceptada con Burp Suite, mostrando el payload inyectado en el campo email.
- Captura de la respuesta HTTP 200 con el token JWT y el correo admin@juice-sh.op en el cuerpo.
- Captura del panel /administration accedido con la sesión obtenida, demostrando la elevación de privilegios.

5.6 Pasos para reproducir

- Paso 1: Acceder a `http://<IP>:3000/#/login` en el navegador.
- Paso 2: En el campo Email ingresar: ' OR 1=1--
- Paso 3: En el campo Password ingresar cualquier valor (por ejemplo: 12345).
- Paso 4: Pulsar el botón Log in.
- Paso 5: Verificar que la barra superior muestra la sesión iniciada como admin@juice-sh.op.
- Paso 6: Navegar a `http://<IP>:3000/#/administration` y confirmar el acceso al panel administrativo.

6. Recomendaciones de Seguridad

6.1 Remediación inmediata (a nivel de código)

Sustituir la consulta SQL construida por concatenación de cadenas por una consulta parametrizada que utilice bind variables. El ORM Sequelize ya está integrado en el proyecto y soporta este mecanismo de manera nativa, por lo que la corrección no requiere agregar dependencias adicionales. La porción de código corregida se muestra en la sección 7 del presente informe y se acompaña de la demostración en vivo durante la sustentación.

6.2 Defensa en profundidad

- Validación de entrada: aplicar validación estricta del formato del campo email mediante expresiones regulares antes de cualquier operación de base de datos.
- Principio de menor privilegio en la base de datos: la cuenta usada por la aplicación para conectarse al motor SQL no debería tener permisos de DDL ni acceso a tablas distintas de las requeridas por el flujo de autenticación.
- WAF: desplegar un Web Application Firewall (AWS WAF, Cloudflare WAF) con reglas específicas contra patrones de inyección SQL.

- Logging y monitoreo: registrar y alertar sobre intentos de login con patrones sospechosos en el campo email (presencia de comillas simples, dobles guiones, palabras clave SQL).
- Pruebas automatizadas en el pipeline: integrar herramientas SAST como Semgrep o SonarQube en CI/CD para detectar este tipo de patrones antes del despliegue.

6.3 Política y cumplimiento

Esta vulnerabilidad debe ser tratada en el marco de la política de desarrollo seguro de la organización y verificada contra los controles del estándar de codificación segura adoptado (por ejemplo, OWASP ASVS V5 — Validation, Sanitization and Encoding). En el contexto de ISO/IEC 27001, el control aplicable es A.8.28 (Codificación segura).

7. Solución Propuesta en Código

7.1 Código vulnerable (estado actual)

Archivo: routes/login.ts. La construcción de la consulta SQL realiza concatenación directa con el valor del campo email proveniente del cliente:

```
models.sequelize.query(
  `SELECT * FROM Users WHERE email = '${req.body.email} || ''}' AND password =
  '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`,
  { model: UserModel, plain: true }
)
```

7.2 Código corregido

La corrección reemplaza la concatenación por marcadores de posición y un arreglo de bind variables. Sequelize se encarga de escapar correctamente los valores antes de pasarlos al motor SQL, neutralizando cualquier intento de inyección:

```
models.sequelize.query(
  `SELECT * FROM Users WHERE email = $email AND password = $password AND
  deletedAt IS NULL`,
  {
    bind: {
      email: req.body.email || '',
      password: security.hash(req.body.password || '')
    },
    model: UserModel,
    plain: true
  }
)
```

Tras aplicar la corrección, el mismo payload "' OR 1=1--" deja de funcionar: Sequelize lo trata como una cadena literal y la consulta busca un usuario cuyo email sea exactamente "' OR 1=1--", el cual no existe. La respuesta del servidor pasa a ser HTTP 401 Unauthorized.

8. Herramientas Utilizadas

- OWASP Juice Shop v17.x — aplicación bajo prueba.
- Docker 24.x — contenedor de la aplicación.
- AWS EC2 (Ubuntu 22.04, t2.medium) — infraestructura del despliegue.
- Burp Suite Community Edition — proxy de interceptación y análisis de peticiones HTTP.
- Visual Studio Code — análisis estático del código fuente y aplicación del parche.
- Navegador Mozilla Firefox — verificación funcional del ataque y de la corrección.

9. Conclusiones

9.1 Fortalezas observadas

- La aplicación implementa autenticación basada en tokens JWT y separa correctamente el frontend (Angular) del backend (Express).
- El proyecto integra el ORM Sequelize, que soporta consultas parametrizadas de manera nativa, lo cual significa que la corrección no requiere refactorizaciones mayores ni nuevas dependencias.
- La aplicación cuenta con un esquema de roles diferenciado (cliente, administrador), lo que demuestra una intención de control de acceso adecuada.

9.2 Debilidades observadas

- El equipo de desarrollo construye consultas SQL críticas mediante concatenación de cadenas, una práctica desaconsejada por todas las guías modernas de codificación segura.
- No se observa validación de formato sobre el campo email antes de su uso en la consulta.
- La arquitectura no incluye una capa de WAF que pudiera detectar y bloquear patrones de inyección antes de que alcancen al backend.

9.3 Conclusión general

La vulnerabilidad encontrada es de severidad crítica y de explotación trivial, pero su corrección es directa y de bajo costo. Una vez aplicado el parche y reiniciado el servicio, la misma carga útil que comprometía el sistema deja de tener efecto. Se recomienda incorporar la verificación de este tipo de patrones en el pipeline de desarrollo mediante herramientas SAST y revisiones de código pareadas, así como complementar la defensa con controles compensatorios a nivel de red y aplicación.